

Методичка: балансировка web-сервисов через HAProxy с health checks

Темы: HAProxy, reverse proxy, frontend, backend, roundrobin, health checks, stats, backend failover, Zabbix

ОС сервера: Debian Server 12/13 или Ubuntu Server 22.04/24.04 LTS

ОС клиента: Debian/Ubuntu или Windows 10/11

Среда: VirtualBox

Формат: самостоятельная практическая работа

Ориентировочное время: 4-6 академических часов

1. Что настроим

В этой работе собираем Linux-сервис или комплексную инфраструктуру. Команды выполняем на Debian/Ubuntu, результат проверяем локально, с клиента и через журналы.

Используем узлы: client01, lb01, web01, web02, zbx01.

Главная идея работы:

HAProxy считается настроенным не тогда, когда процесс запущен, а тогда, когда клиент получает ответы через lb01, запросы распределяются между backend, отказ одного backend не ломает сервис, а stats показывают состояние серверов.

2. Схема учебного стенда

Узел	Роль	ОС	Сетевые адаптеры	IP-адрес
lb01	балансировщик или входной узел	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.10 /24
web01	backend/web/mail/service	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.21 /24
web02	backend или дополнительный узел	Debian/Ubuntu Server	NAT + внутренняя сеть	192.168.10.22 /24
client01	клиент проверки	Debian/Ubuntu или Windows	внутренняя сеть	192.168.10.30 /24

```
client01 -> lb01/service entry -> web01/web02/mail/monitoring/backup
```

NAT используем для установки пакетов. Сервисы проверяем во внутренней сети. Имена добавляем через DNS или `/etc/hosts` и проверяем `getent hosts`.

3. Необходимые ресурсы и предварительные условия

Нужны:

- lb01 для HAProxy;
- web01 и web02 с Nginx и разными страницами;
- клиент для HTTP-проверки;
- пакеты haproxy, nginx, curl;
- доступ к Zabbix, если проверяется мониторинг.

4. Подготовка виртуальных машин

```
hostnamectl  
ip -br address  
ip route  
resolvectl status  
systemctl --failed
```

Проверяем имена:

```
getent hosts lb01.company.test  
getent hosts web01.company.test  
getent hosts web02.company.test
```

Самопроверка этапа

- узлы включены;
- адреса и имена согласованы;
- NAT доступен для установки пакетов;
- снимки VM созданы.

Если результат отличается от ожидаемого, дальше пока не продолжаем. Сначала проверяем этот этап.

5. Адресный план или план ресурсов

Ресурс	Значение
Входной узел	lb01, 192.168.10.10
Backend 1	web01, 192.168.10.21
Backend 2	web02, 192.168.10.22
Клиент	client01, 192.168.10.30
Учебный домен	company.test

План работы:

- подготовить разные страницы на web01 и web02;
- установить HAProxy на lb01;
- настроить frontend и backend roundrobin;
- включить health checks;
- проверить распределение запросов;
- остановить один backend и проверить исключение из балансировки;
- включить stats и подключить мониторинг.

6. Исходная проверка

```
cat /etc/os-release
ip -br address
ip route
ss -lntup
systemctl --failed
```

```
df -h
```

Самопроверка этапа

- сеть работает;
- нужные порты свободны или понятны;
- свободного места достаточно;
- failed-службы разобраны.

7. Пошаговая настройка

7.1. Подготовка пакетов

```
sudo apt update
```

Если пакеты не скачиваются, проверяем NAT, DNS и маршрут.

7.2. Основная настройка

На backend-серверах:

```
sudo apt install nginx -y  
echo "web01" | sudo tee /var/www/html/index.html  
sudo systemctl enable --now nginx
```

На web02 пишем web02 в index.html.

На lb01:

```
sudo apt install haproxy -y  
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.bak  
sudo nano /etc/haproxy/haproxy.cfg
```

Фрагмент:

```
frontend http_in  
    bind *:80  
    default_backend web_backends  
  
backend web_backends  
    balance roundrobin
```

```
option httpchk GET /  
server web01 192.168.10.21:80 check  
server web02 192.168.10.22:80 check  
  
listen stats  
  bind *:8404  
  stats enable  
  stats uri /stats
```

Проверяем и применяем:

```
sudo haproxy -c -f /etc/haproxy/haproxy.cfg  
sudo systemctl restart haproxy
```

Самопроверка этапа

```
systemctl status haproxy --no-pager  
ss -lntup | grep haproxy  
for i in 1 2 3 4; do curl -s http://192.168.10.10; done  
curl -I http://192.168.10.10:8404/stats  
sudo journalctl -u haproxy -n 50 --no-pager
```

Останавливаем Nginx на `web01` и повторяем curl: ответы должны продолжаться с `web02`.

Если результат отличается от ожидаемого, дальше пока не продолжаем.

Сначала проверяем этот этап.

7.3. Восстановление или откат

Проверяем откат: возвращаем backend, восстанавливаем контейнер, проверяем бэкап или возвращаем конфигурацию из `.bak`. Для сложных сервисов сначала восстанавливаем тестовый объект, а не весь сервер.

8. Проверка итогового результата

```
systemctl status haproxy --no-pager  
ss -lntup | grep haproxy  
for i in 1 2 3 4; do curl -s http://192.168.10.10; done  
curl -I http://192.168.10.10:8404/stats  
sudo journalctl -u haproxy -n 50 --no-pager
```

Останавливаем Nginx на `web01` и повторяем curl: ответы должны продолжаться с `web02`.

Границы результата:

- учебная настройка не заменяет промышленную HA-архитектуру;
- публичная почта требует реального DNS, PTR, репутации IP и TLS;
- тестовые пароли, DKIM-ключи и секреты нельзя публиковать;
- сервис считается готовым только после клиентской проверки и журналов.

9. Диагностика типовых ошибок

Симптом	Вероятная причина	Проверка	Исправление
HAProxy не стартует	Ошибка синтаксиса	<code>haproxy -c -f /etc/haproxy/haproxy.cfg</code>	Исправить строку из вывода
Ответ всегда от одного backend	Второй backend down или health check падает	<code>stats, curl web02</code>	Проверить Nginx и адрес

Stats недоступен	Не настроен listen stats или порт закрыт	<code>ss -lntup</code> , <code>curl</code>	Исправить bind и firewall
Порт занят	Другая служба уже слушает	<code>ss -lntup</code>	Освободить порт или изменить конфигурацию
Клиент не подключается	DNS, firewall или служба	<code>getent hosts</code> , <code>nc -vz</code> , <code>journalctl</code>	Исправить имя, порт или службу
Диск заполнен	Логи, mail queue, Docker volumes или backup	<code>df -h</code> , <code>du -xhd1</code>	Очистить или расширить хранилище по плану

10. Итоговая самостоятельная проверка

- [] Стенд соответствует схеме.

- [] Имена и адреса согласованы.
- [] Пакеты или контейнеры установлены.
- [] Основной сервис запущен.
- [] Порт слушается.
- [] Клиентская проверка проходит.
- [] Отказ или восстановление проверены.
- [] В журналах нет необъяснённых ошибок.
- [] Одна типовая ошибка найдена и исправлена.

11. Что приложить к отчёту

К отчёту прикладываем схему, адресный план, конфигурацию, вывод проверки службы/контейнеров, клиентский результат, журнал, результат отказового теста или восстановления, описание исправленной ошибки.

12. Контрольные вопросы

1. Какую задачу решает технология этой лекции?
2. Какие зависимости нужно проверить до запуска сервиса?
3. Какая команда подтверждает основной результат?
4. Как проверить сервис с клиента?
5. Что искать в журналах?
6. Как выполняется восстановление?
7. Какие ограничения есть у учебного стенда?

8. Что обязательно документировать?